Design and Implementation of a Highly Available AWS Web Infrastructure

A Cloud Infrastructure Solution for Meridian Retail Solutions (MRS)

Project Type	**Personal Hands-on Cloud Infrastructure Project**
Business Scenario	Meridian Retail Solutions (Fictional Medium-Sized Enterprise)
Cloud Platform	Amazon Web Services (AWS)
AWS Region	Asia Pacific (Singapore) — ap-southeast-1
Project Domain	Cloud Infrastructure, Networking, Security and High Availability
Technologies	Amazon VPC, EC2, ALB, Auto Scaling, IAM, Systems Manager, CloudWatch, SNS, NAT Gateway

Prepared By : Chanmith Jayathilake

Role : Cloud Infrastructure Engineer (Personal Project)

Project Status : Completed

Documentation Date : October 2026

Copyright Notice

© 2026 P.C.L.D Jayathilake. All rights reserved.

This documentation is the original intellectual property of the author. It is protected under applicable national and international copyright laws.

No part of this document may be copied, reproduced, distributed, published, transmitted, modified, translated, or stored in any form or by any means-electronic, mechanical, photocopying, recording, or otherwise-without the prior written permission of the author, except for brief quotations used for educational or non-commercial purposes with proper attribution.

Unauthorized use, duplication, or distribution of this material without permission may result in legal action.

The author retains full moral and intellectual property rights over this work.

Ethical Declaration of Original Work and Authorization

I hereby formally declare that this documentation is an original work created by me and represents my independent effort, knowledge, and practical understanding of the subject matter. All concepts, configurations, explanations, diagrams, workflows, and technical descriptions included in this document have been developed based on my academic studies, professional training, hands-on practice, and personal analysis.

I confirm that this work has not been copied, duplicated, or plagiarized from any other source. Where external materials, standards, documentation, tools, or references have been consulted for learning or verification purposes, they have been appropriately acknowledged and credited in a fair and ethical manner. No part of this document intentionally infringes upon the intellectual property rights of any individual, organization, or institution.

This documentation has been prepared in accordance with ethical, academic, and professional standards. It is intended solely for educational, informational, and professional knowledge-sharing purposes. Any similarities to existing works are purely coincidental or arise from commonly accepted technical practices and industry-standard terminology.

Authorization Statement

I, the undersigned author, retain full ownership and intellectual property rights over this documentation. I authorize the public viewing and reading of this work through approved platforms where it is published or shared by me.

Unless explicitly stated otherwise, no individual, organization, or third party is permitted to copy, reproduce, modify, translate, redistribute, sell, or use this material for commercial purposes without my prior written consent.

Limited use of this documentation is authorized for educational and non-commercial purposes only, provided that:

- Proper credit is clearly given to the author
- The content is not altered in a way that misrepresents the original work
- The use does not violate ethical, legal, or professional standards

Any unauthorized reproduction, distribution, or misuse of this material may result in legal or ethical consequences in accordance with applicable intellectual property laws.

I accept full responsibility for the accuracy, integrity, and originality of the content presented in this documentation.

Field: Network Engineering & Ethical Hacking
Date: October 2026

TABLE OF CONTENTS

1. Introduction
2. Problem Domain
3. Proposed Solution
4. Project Aim
5. Objectives
6. Business Requirements
7. System & Network Architecture and Design
    - 7.1 Target Architecture
    - 7.2 IP Address Allocation
    - 7.3 Routing and Segmentation
    - 7.4 Availability boundary and production considerations
8. Implementation Technologies
9. Configurations
    - 9.1 Select region and configure cost alert
    - 9.2 Configure cost alert
    - 9.3 Create the VPC (Virtual Private Cloud)
    - 9.4 Create four subnets
    - 9.5 Configure Internet Gateway
    - 9.6 Configure Public routing
    - 9.7 Public Subnet Associations
    - 9.8 Configure NAT Gateway
    - 9.9 Configure Private Routing
    - 9.10 Private Subnet Association
    - 9.11 Security Group Creation – ALB
    - 9.12 Security Group Creation – EC2
    - 9.13 Configure IAM role for secure host management
    - 9.14 Build the EC2 launch template
    - 9.15 Nginx installation with User-data
    - 9.16 Configure target group and health checks
    - 9.17 Deploy internet-facing Application Load Balancer
    - 9.18 Deploy Auto Scaling Group
    - 9.19 Two Healthy Targets (EC2s)
    - 9.20 Validate web access and instance isolation
    - 9.21 Verifying EC2 Instances have No Public IP Addresses
    - 9.22 Verify AWS Systems Manager access
    - 9.23 Configure SNS email alerting
    - 9.24 Configure CloudWatch monitoring and Dashboard
    - 9.25 Auto Scaling and Automated Instance Recovery Testing
10. References

1. INTRODUCTION

Meridian Retail Solutions (MRS) is a fictional medium-sized company operating regional retail and distribution services. Its public web presence is assumed to support product information, customer enquiries and operational communication. For this case study, MRS requires a resilient web-hosting foundation that can be managed by a small infrastructure team without maintaining exposed administrative access to individual servers.

This personal AWS project implements the infrastructure foundation for that business scenario in the Singapore AWS Region. It demonstrates network segmentation, Amazon EC2 hosting, Application Load Balancer routing, EC2 Auto Scaling instance replacement, IAM-based server management and CloudWatch operational visibility. A simple Nginx website acts as a test workload; building application features is outside the scope.

The implementation is a short-duration demonstration environment rather than a full production release. The lab intentionally uses HTTP, a single shared NAT Gateway and fixed two-instance capacity. These constraints are recorded explicitly so the report does not overstate production readiness.

2. PROBLEM DOMAIN

Meridian Retail Solutions (MRS), a medium-sized business, faces several infrastructure challenges that can affect website availability, security, reliability, and operational efficiency.

1. Single Point of Failure: Hosting a website on a single server creates a risk of service interruption if the server crashes, becomes unhealthy, or requires maintenance.

2. Limited High Availability: Without infrastructure distributed across multiple Availability Zones, server or Availability Zone failures can disrupt business operations and website accessibility.

3. Network Security Risks: Exposing web servers directly to the public internet increases the potential attack surface and risk of unauthorized access.

4. Manual Failure Recovery: Traditional infrastructure requires administrators to identify failures, restart services, or replace servers manually, increasing downtime and operational workload.

5. Inefficient Traffic Distribution: Without load balancing, incoming traffic cannot be distributed across multiple web servers, potentially causing performance issues and uneven resource utilization.

6. Limited Monitoring and Alerting: Without centralized monitoring and automated notifications, administrators may struggle to detect server failures, performance degradation, and infrastructure health issues promptly.

7. Complex Infrastructure Management: Managing network configurations, server access, security policies, and recovery procedures manually increases administrative complexity and the risk of configuration errors.

8. Business Continuity Challenges: Unexpected infrastructure failures can interrupt customer access, reduce service reliability, and negatively affect business operations.

3. PROPOSED SOLUTION

To address the identified infrastructure challenges, Meridian Retail Solutions (MRS) proposes the implementation of a Highly Available, Secure, and Automatically Recoverable AWS Web Infrastructure using the following solutions:

1. Elimination of Single Points of Failure: Deploy Amazon EC2 web servers across two AWS Availability Zones to reduce dependency on a single server and improve service availability.

2. High Availability and Fault Tolerance: Implement an Application Load Balancer (ALB) to distribute incoming HTTP traffic across healthy EC2 instances in both Availability Zones, maintaining website accessibility during individual server failures.

3. Enhanced Network Security: Configure a dedicated Amazon VPC (10.10.0.0/16) in the Singapore Region (ap-southeast-1) with two public subnets and two private subnets. Deploy the ALB in public subnets and EC2 instances in private subnets to prevent direct internet access to backend servers.

4. Automated Failure Recovery: Implement Amazon EC2 Auto Scaling with a launch template and Elastic Load Balancing health checks to detect unhealthy instances and automatically replace them, maintaining a desired capacity of two EC2 instances.

5. Efficient Traffic Distribution: Configure an internet-facing Application Load Balancer with an HTTP target group to distribute incoming requests between available web servers and route traffic only to healthy targets.

6. Centralized Monitoring and Alerting: Utilize Amazon CloudWatch to monitor EC2 resource utilization, load balancer health, and infrastructure performance. Integrate Amazon Simple Notification Service (SNS) to deliver automated email notifications when configured alarm thresholds are breached.

7. Simplified and Secure Infrastructure Management: Implement AWS Identity and Access Management (IAM) roles and AWS Systems Manager Session Manager to securely manage private EC2 instances without exposing SSH port 22 to the public internet.

8. Improved Business Continuity and Operational Reliability: Configure security groups, public and private route tables, an Internet Gateway, and a NAT Gateway to control network communication and enable outbound connectivity for private instances. Combine these controls with automatic instance replacement and monitoring to reduce manual recovery efforts and improve service reliability.

Proposed Network Configuration

Network Component	Configuration
AWS Region	Asia Pacific (Singapore) — ap-southeast-1
VPC	10.10.0.0/16
Public Subnet A	10.10.10.0/24
Public Subnet B	10.10.20.0/24
Private Subnet A	10.10.30.0/24
Private Subnet B	10.10.40.0/24
Application Load Balancer	Internet-facing, HTTP port 80
Auto Scaling Group	Minimum: 2, Desired: 2, Maximum: 2
Private Subnet Internet Access	Single NAT Gateway in Public Subnet A
Secure Administration	IAM Roles and Systems Manager Session Manager
Monitoring and Notifications	Amazon CloudWatch and Amazon SNS

4. PROJECT AIM

To design, configure and evaluate an AWS-hosted web infrastructure prototype for a medium-sized business that demonstrates increased server availability, private network segmentation, controlled administrative access, automated instance recovery and operational monitoring using managed AWS services.

5. OBJECTIVES

1. Design a logically isolated VPC with four correctly routed subnets across two Availability Zones.
2. Expose only an Application Load Balancer to public HTTP traffic while keeping EC2 servers private.
3. Configure a reproducible EC2 launch template with Amazon Linux 2023 and Nginx bootstrap user data.
4. Operate two EC2 instances through an Auto Scaling Group with load balancer health checks.
5. Restrict server ingress via security group references and manage instances using Session Manager and an IAM role.
6. Create CloudWatch infrastructure visibility and SNS-backed notifications for unhealthy targets.
7. Exercise a controlled instance-failure scenario and capture replacement/recovery evidence.
8. Document configuration choices, technical limitations, cost considerations and safe resource teardown.

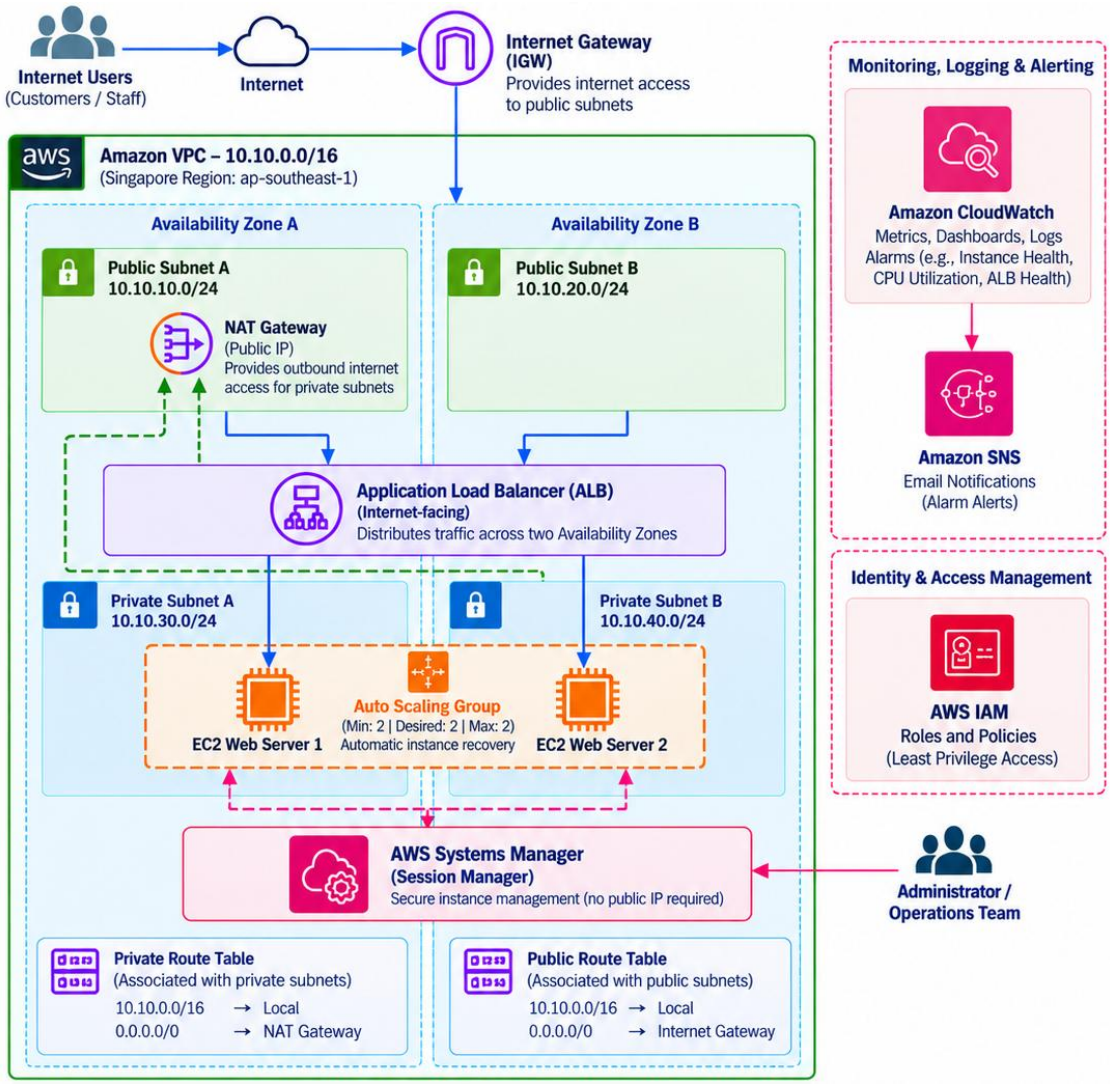
6. BUSINESS REQUIREMENTS

The following business requirements define the operational, security, availability, and management needs of Meridian Retail Solutions (MRS) for implementing a Highly Available AWS Web Infrastructure.

Requirement ID	Business Requirement	Description	Proposed AWS Solution
BR-01	High Availability	Ensure the business website remains accessible during individual server failures.	EC2 instances across two Availability Zones.
BR-02	Fault Tolerance	Minimize service interruptions by distributing incoming requests across healthy web servers.	Application Load Balancer (ALB) with target group health checks.
BR-03	Automated Failure Recovery	Automatically replace unhealthy or failed servers without manual intervention.	EC2 Auto Scaling Group with launch templates and health checks.
BR-04	Network Security	Protect backend web servers from direct public internet access.	Amazon VPC, public/private subnets, security groups, and route tables.
BR-05	Secure Administration	Enable authorized administrators to securely manage private EC2 instances without exposing SSH access.	AWS IAM roles and Systems Manager Session Manager.
BR-06	Traffic Distribution	Efficiently distribute incoming web traffic between available servers.	Application Load Balancer with HTTP listener and target group.
BR-07	Infrastructure Monitoring	Monitor server performance, resource utilization, and infrastructure health.	Amazon CloudWatch metrics, alarms, and dashboards.
BR-08	Automated Notifications	Notify administrators when infrastructure failures or abnormal conditions are detected.	CloudWatch Alarms integrated with Amazon SNS email notifications.
BR-09	Controlled Internet Connectivity	Allow private servers to access external services and download required software updates without accepting unsolicited inbound internet connections.	NAT Gateway, Internet Gateway, and configured route tables.
BR-10	Operational Efficiency	Reduce manual infrastructure management and recovery activities.	EC2 Auto Scaling, Systems Manager, and centralized monitoring.
BR-11	Business Continuity	Reduce the impact of individual server failures on website availability and daily business operations.	Multi-AZ deployment, load balancing, and automatic EC2 instance replacement.
BR-12	Cost Management	Maintain visibility over AWS infrastructure expenditure and avoid unnecessary resource consumption.	AWS Budgets, controlled resource provisioning, and resource cleanup procedures.

7. SYSTEM & NETWORK ARCHITECTURE AND DESIGN

7.1 Target Architecture



Logical request path: Internet users → public ALB → healthy HTTP target group → Nginx EC2 web servers in private subnets.

Management path: AWS operator → Systems Manager Session Manager → managed EC2.

Outbound path: private EC2 → shared NAT Gateway in public subnet A → Internet Gateway.

7.2 IP Address Allocation

Network component	CIDR	Zone / purpose
VPC / vpc-enterprise-web	10.10.0.0/16	Dedicated AWS network
subnet-public-ingress-a	10.10.10.0/24	Availability Zone A; ALB and NAT gateway
subnet-public-ingress-b	10.10.20.0/24	Availability Zone B; ALB
subnet-private-web-a	10.10.30.0/24	Availability Zone A; EC2 workload
subnet-private-web-b	10.10.40.0/24	Availability Zone B; EC2 workload

7.3 Routing and Segmentation

Route table	Subnet association	Non-local route
rtb-public-ingress	Both public subnets	0.0.0.0/0 → igw-enterprise-web
rtb-private-web	Both private subnets	0.0.0.0/0 → nat-private-egress-a

Each subnet additionally uses the local VPC route for communication within 10.10.0.0/16. Route-table association determines public versus private egress behavior; private instances do not receive public IPv4 addresses. The one-NAT design introduces cross-AZ dependency for private outbound access and is not equivalent to independent per-AZ NAT resiliency.

7.4 Availability boundary and production considerations

The design demonstrates tolerance to an individual EC2 instance failure after successful replacement, assuming remaining capacity and service health. It does not establish a measured availability SLA, zero-downtime guarantee, disaster recovery plan or fully fault-isolated outbound internet path.

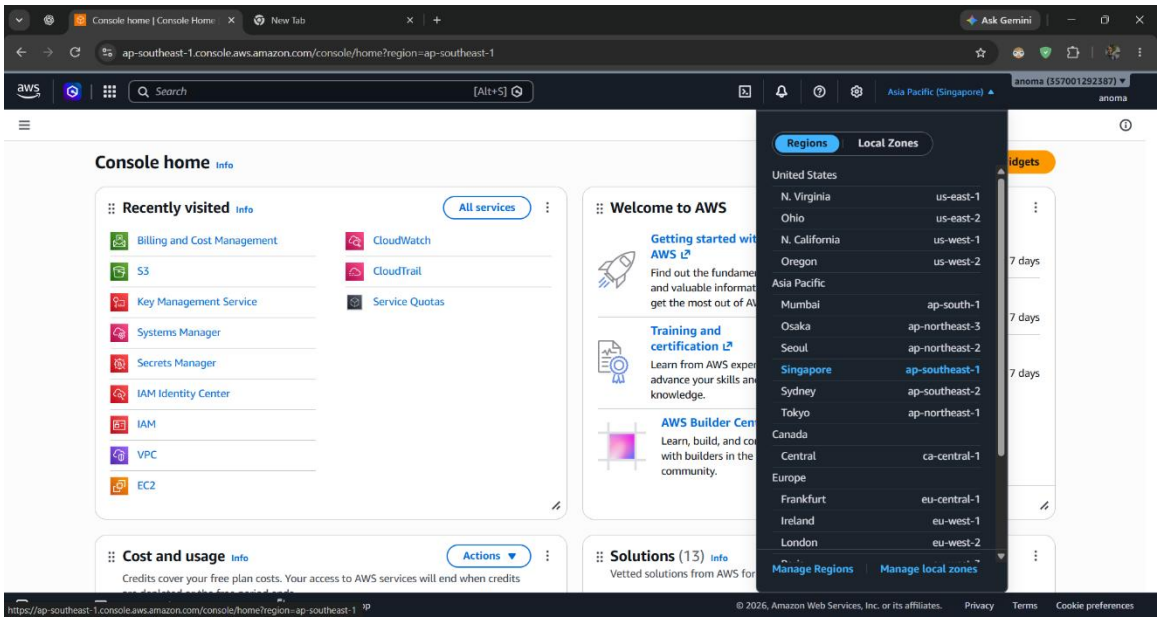
Recommended production improvements include ALB HTTPS with ACM, NAT per AZ or relevant VPC endpoints, scaling policies tied to demand, centralized log retention, stronger egress restrictions, and formal performance/security testing.

8. IMPLEMENTATION TECHNOLOGIES

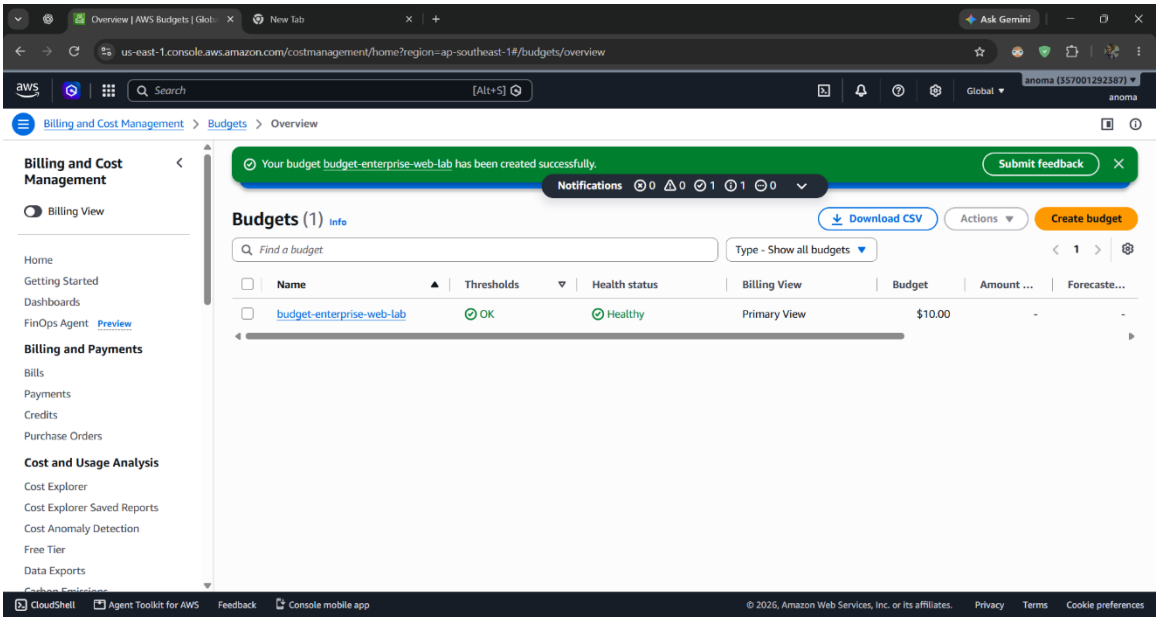
Service / Technology	Function in Project
Amazon VPC, Subnets, Routes, IGW	Logical network boundary, separation of ingress and web tiers, internet routing.
NAT Gateway + Elastic IP	Outbound package installation and SSM connectivity for private instances.
Amazon EC2 + Amazon Linux 2023	Private Linux compute hosting the sample web workload.
Nginx + EC2 user data	Simple HTTP test site and initial software configuration at launch.
Application Load Balancer + Target Group	Single public web entry point, traffic distribution and HTTP health assessment.
EC2 Auto Scaling + Launch Template	Maintains a defined fleet capacity and replaces failing EC2 instances.
AWS IAM + Instance Profile	Delegates SSM managed-instance permissions through AmazonSSMManagedInstanceCore.
AWS Systems Manager Session Manager	Browser-based managed host access without inbound SSH.
Amazon CloudWatch	Metrics, dashboard and unhealthy-host alarm.
Amazon SNS	Email alert delivery through confirmed subscription.
AWS Budgets	Spend alert, not an automatic charge cutoff.

9. CONFIGURATIONS

9.1 Select Region and Configure Cost Alert

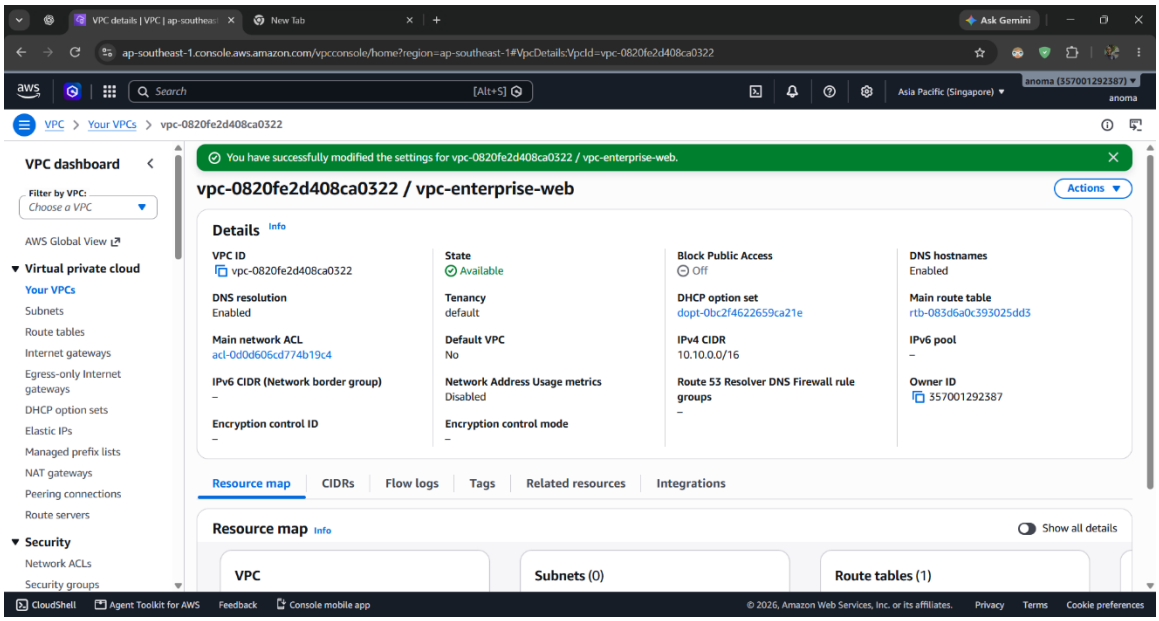


9.2 Configure Cost Alert



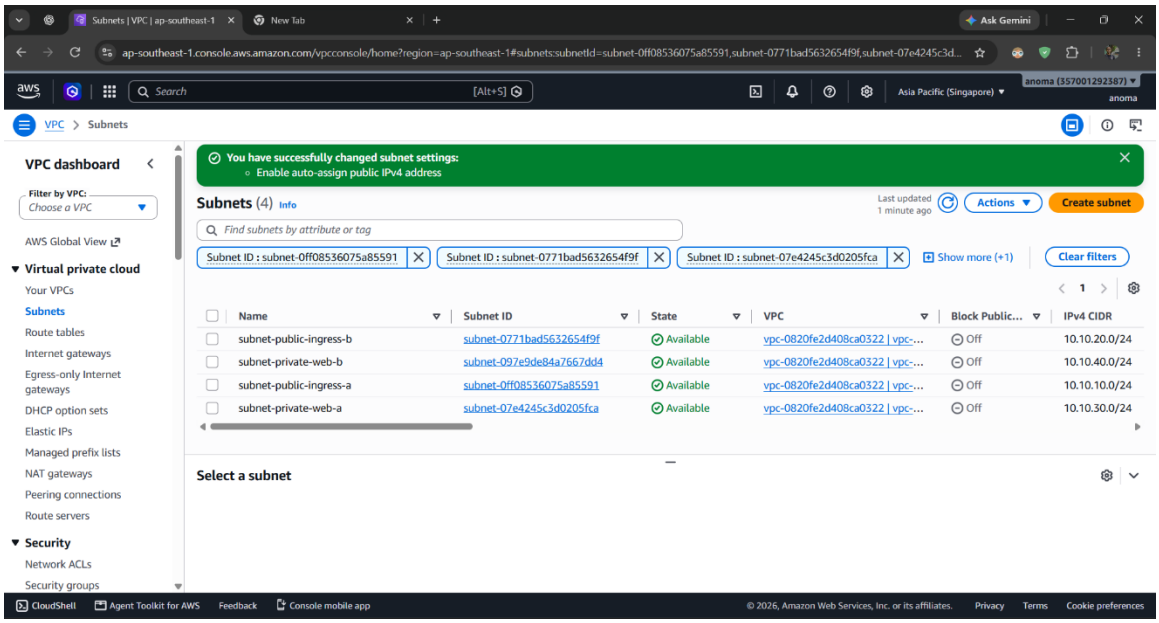
9.3 Create the VPC (Virtual Private Cloud)

Created vpc-enterprise-web with CIDR 10.10.0.0/16; enabled VPC DNS resolution and DNS hostnames.



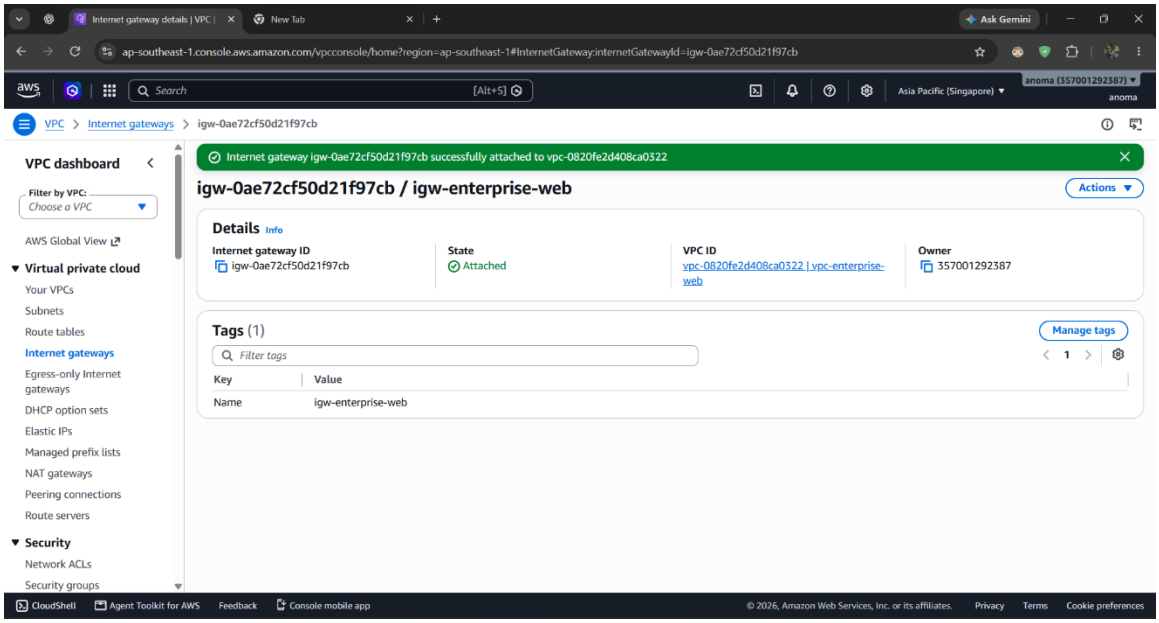
9.4 Create Four Subnets

Created public ingress subnets 10.10.10.0/24 and 10.10.20.0/24, plus private web subnets 10.10.30.0/24 and 10.10.40.0/24, across two distinct AZs. Public auto-assignment was enabled only where appropriate.



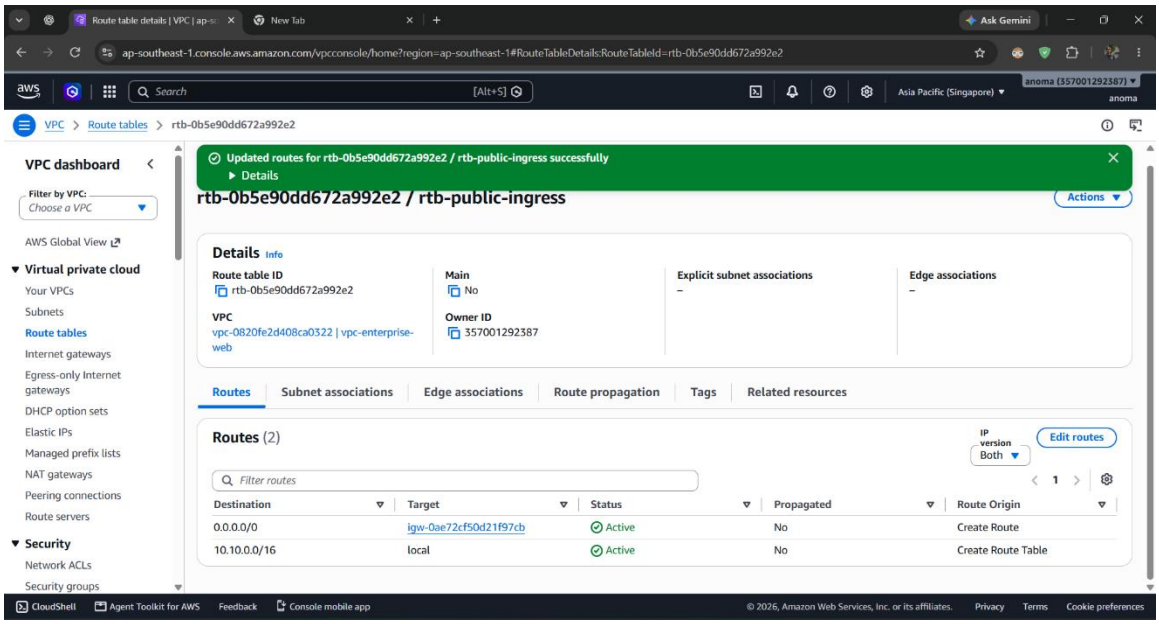
9.5 Configure Internet Gateway

Attached igw-enterprise-web to the VPC.

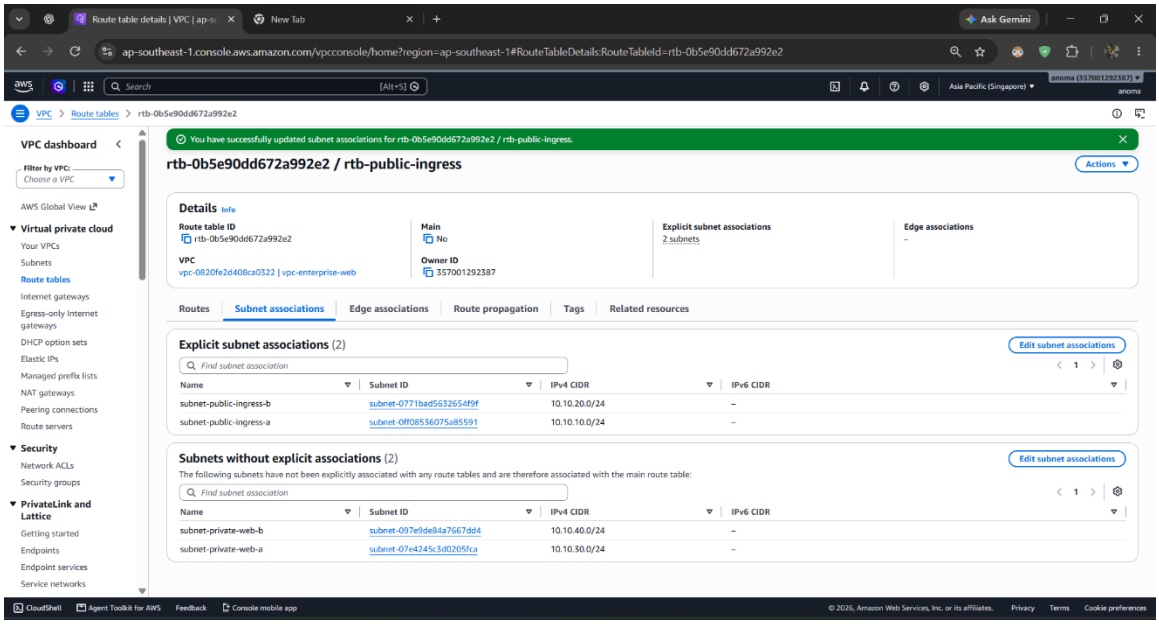


9.6 Configure Public Routing

Created rtb-public-ingress and mapped the two public subnets to default route 0.0.0.0/0 via the IGW.

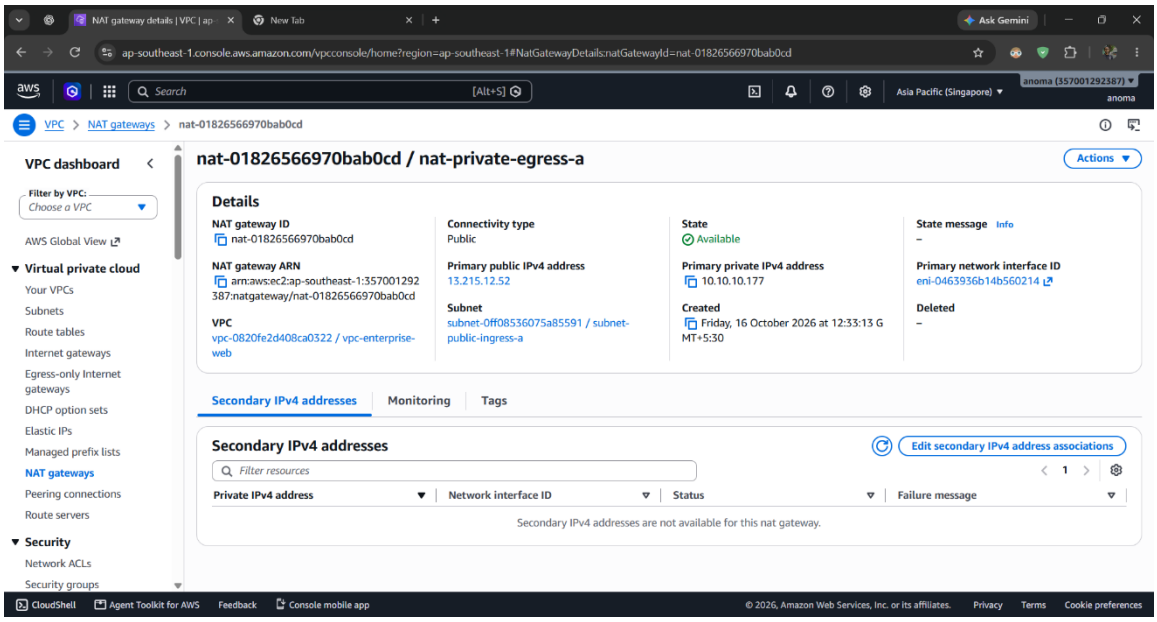


9.7 Public Subnet Associations



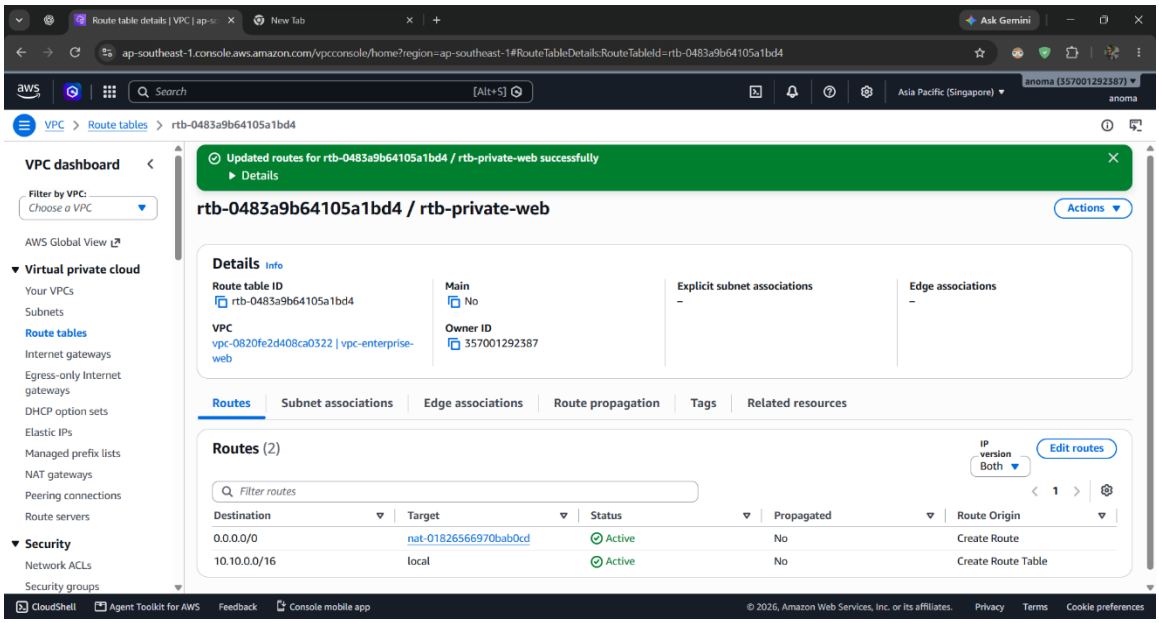
9.8 Configure NAT Gateway

Provisioned public NAT gateway nat-private-egress-a with Elastic IP in public subnet A.

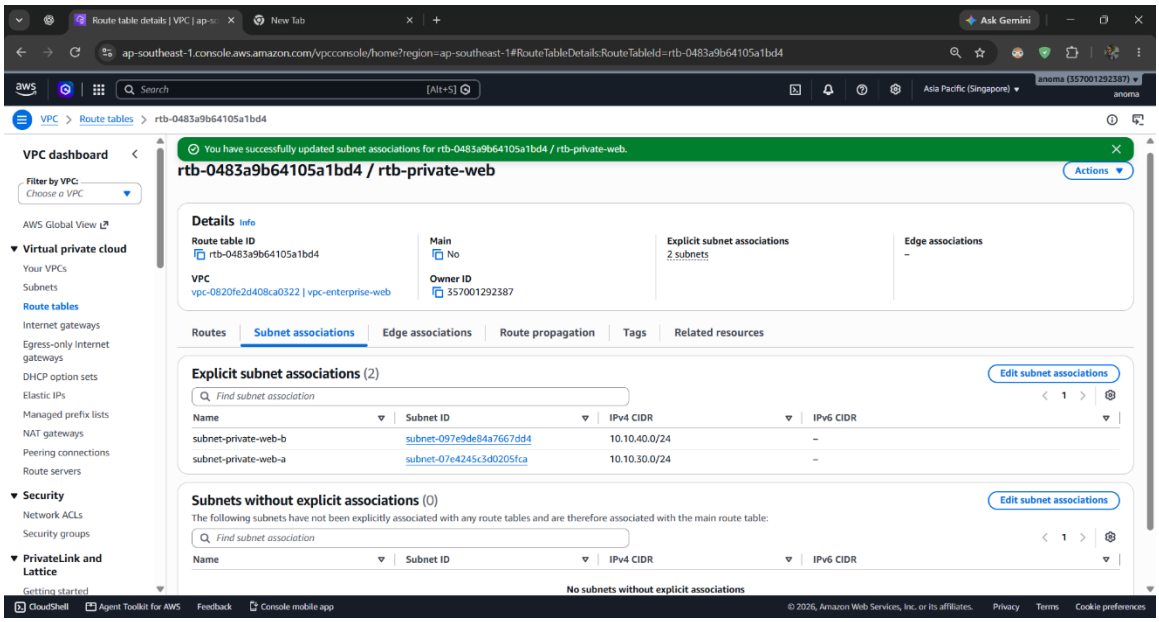


9.9 Configure Private Routing

Mapped both private subnets to rtb-private-web, default route 0.0.0.0/0 via the NAT Gateway.

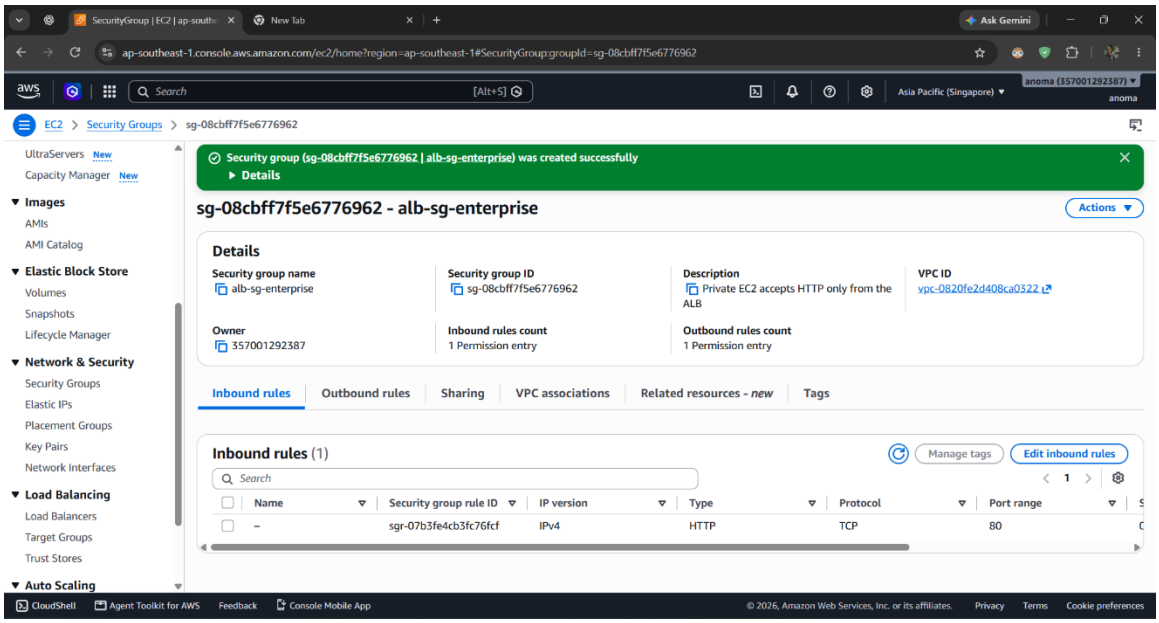


9.10 Private Subnet Association



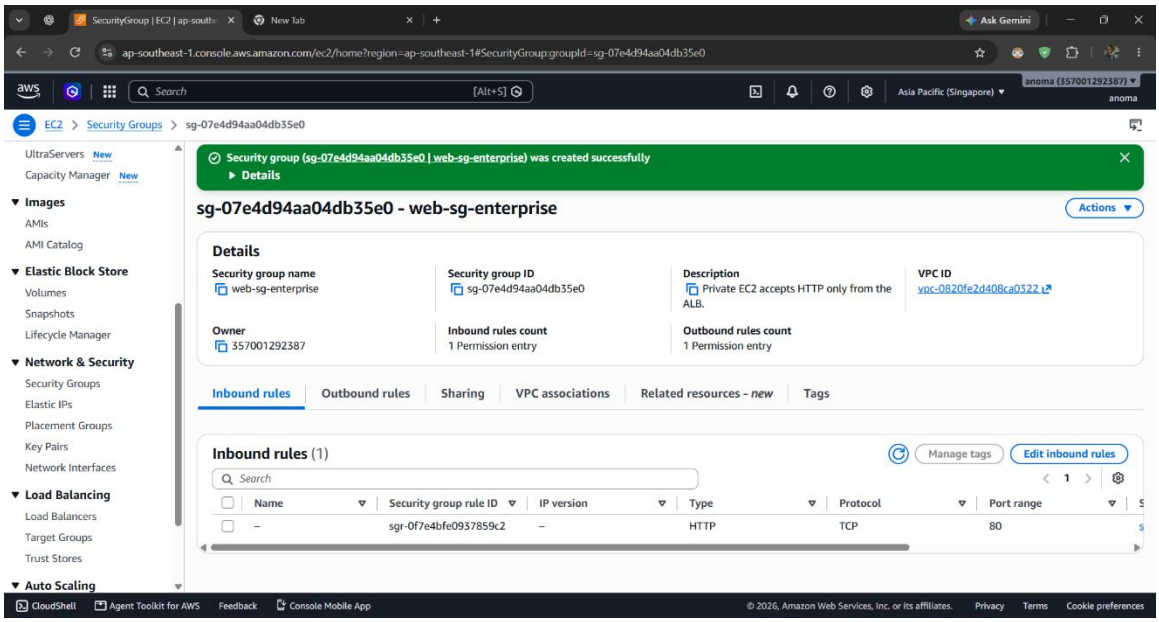
9.11 Security Group Creation – ALB

Created alb- sg-enterprise for internet HTTP access on TCP 80



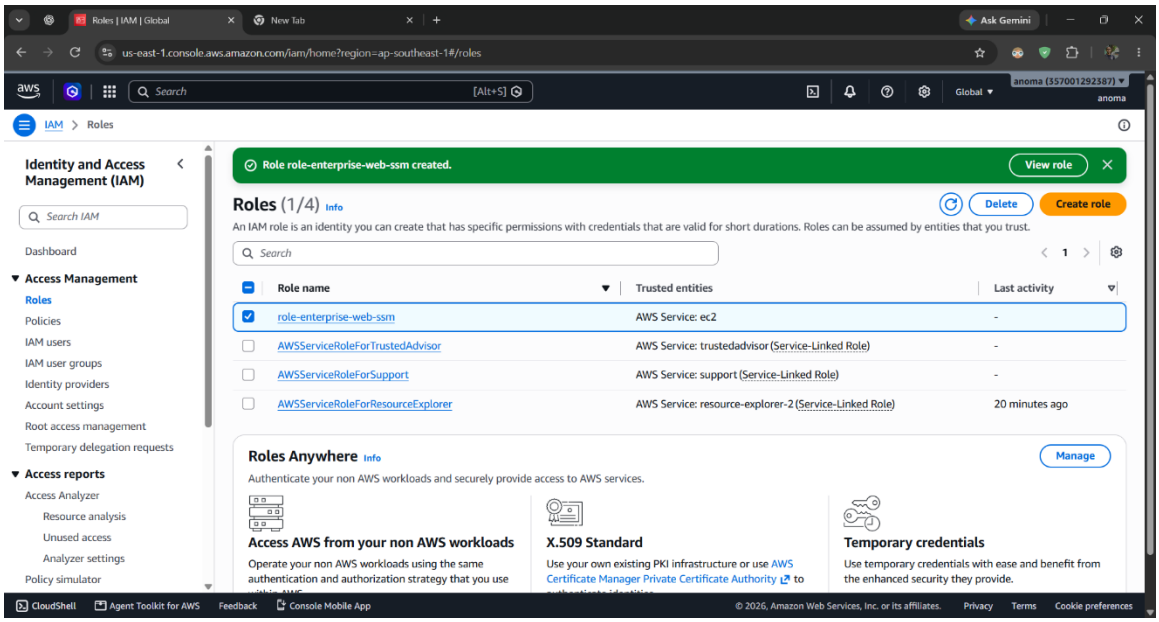
9.12 Security Group Creation – EC2

Web-sg-enterprise for backend TCP 80 sourced exclusively from the ALB SG. Did not open inbound TCP 22.



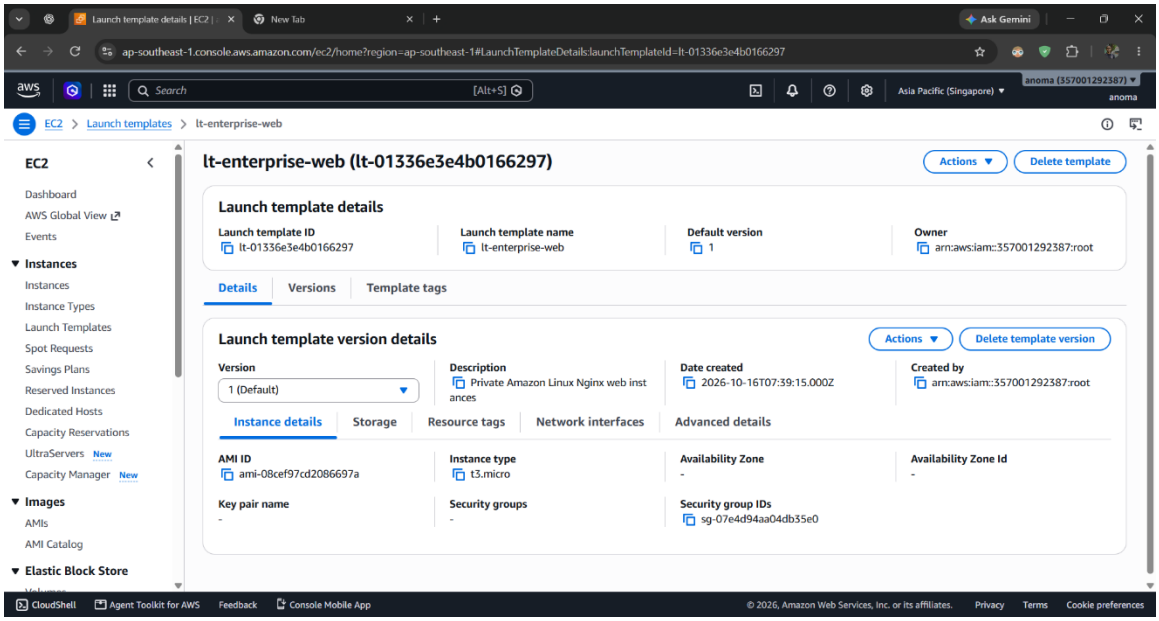
9.13 Configure IAM Role For Secure Host Management

Created role-enterprise-web-ssm trusted by EC2; attached AmazonSSMManagedInstanceCore to support Systems Manager sessions.



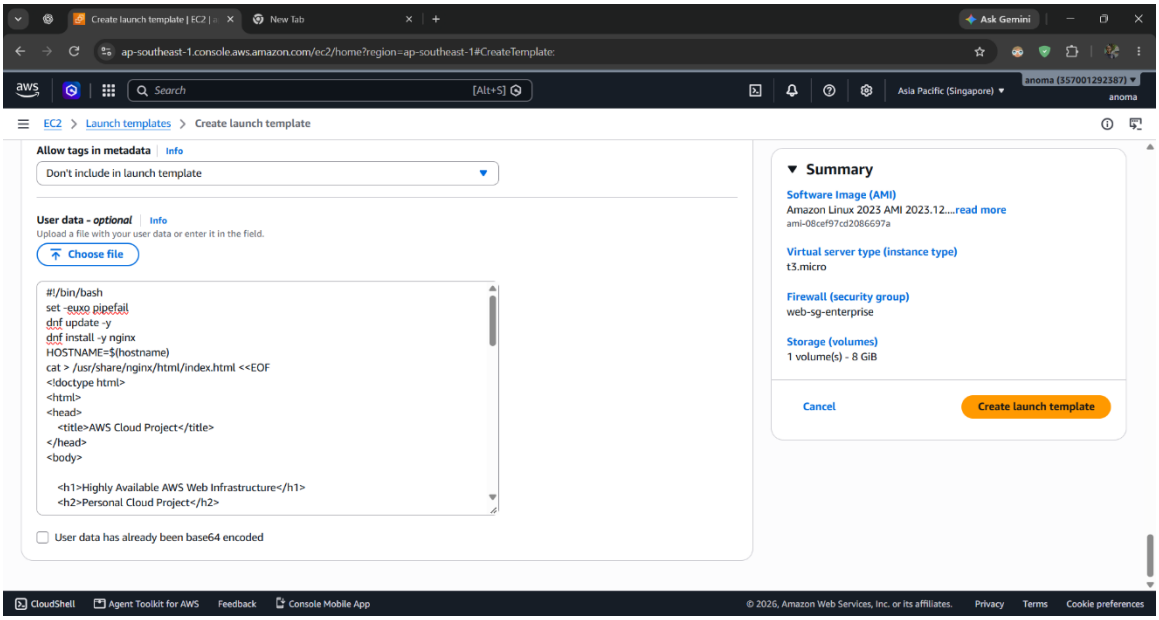
9.14 Build the EC2 Launch Template

Created lt-enterprise-web using Amazon Linux 2023, t3.micro, gp3 8-GiB boot volume, IAM instance profile



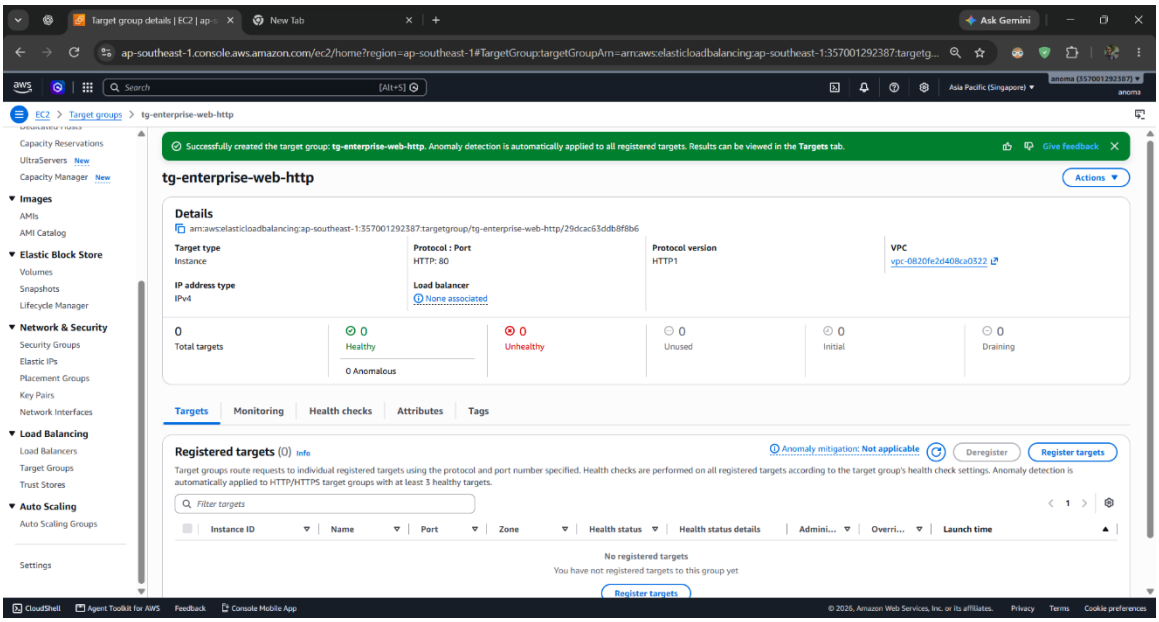
9.15 Nginx Installation with User-data

Bootstrap user-data commands that install and start Nginx. Instances were configured without public IPv4 addressing.



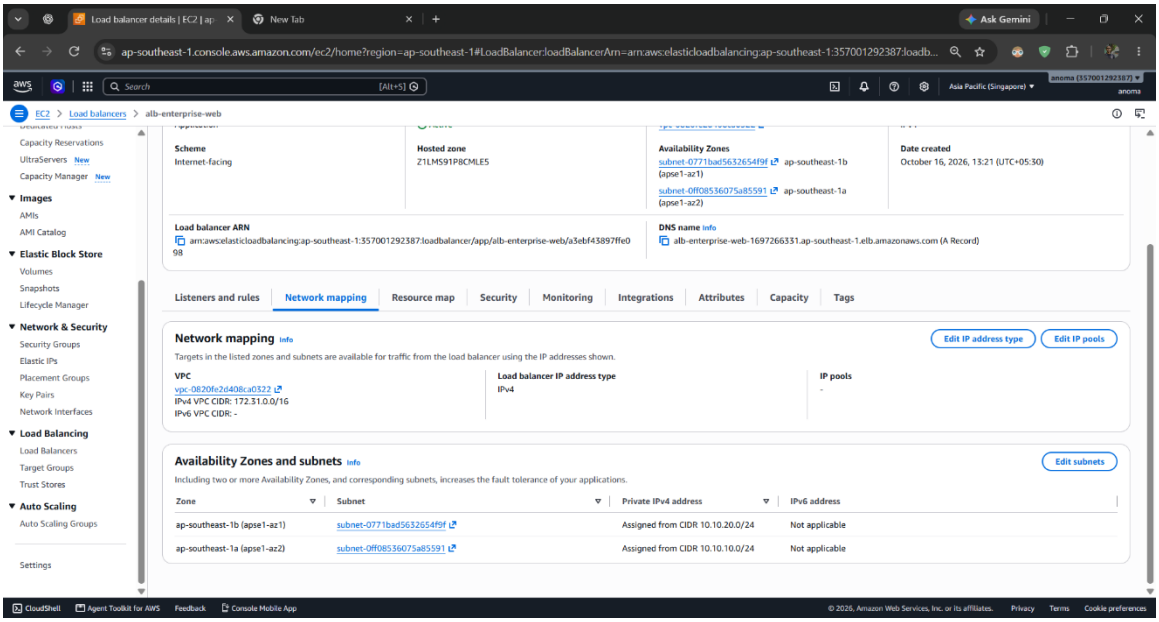
9.16 Configure Target Group and Health Checks

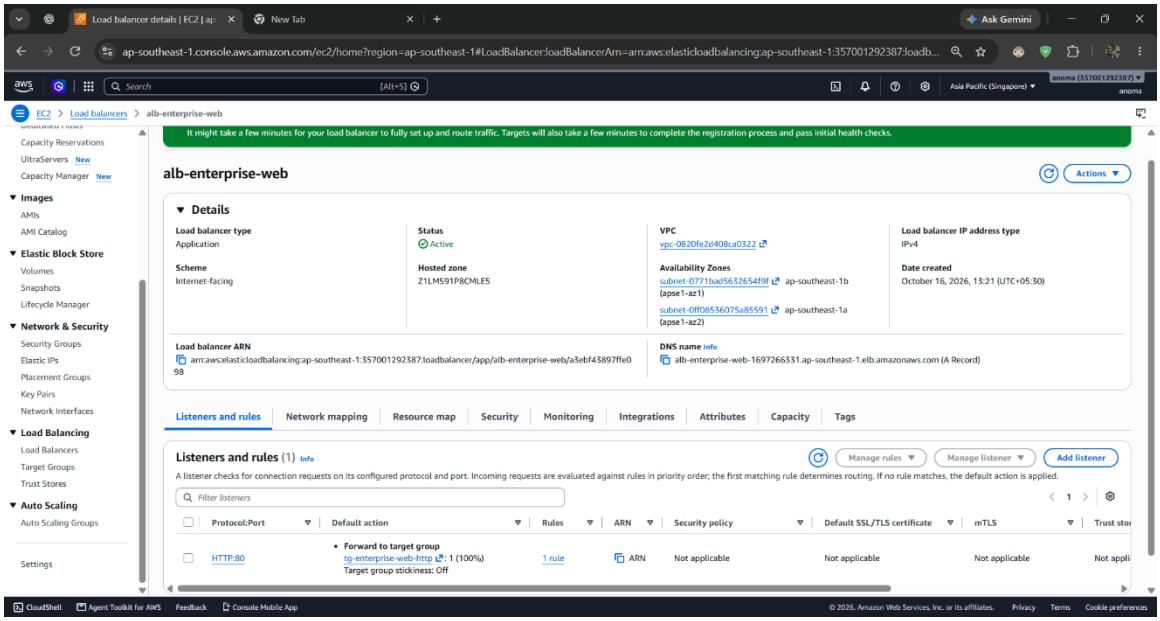
Created tg-enterprise-web-http as an HTTP instance target group on port 80 with GET / health checks and HTTP 200 as accepted success code.



9.17 Deploy Internet-Facing Application Load Balancer

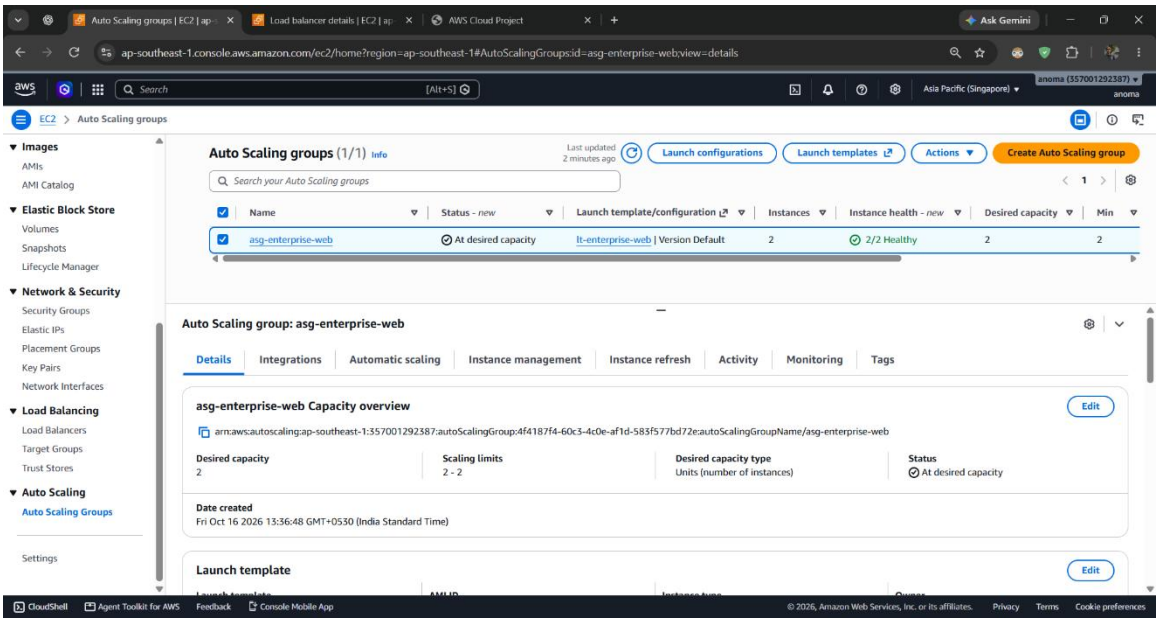
Created alb-enterprise-web across the two public subnets, with alb- sg-enterprise and HTTP port 80 listener forwarding traffic to the target group.



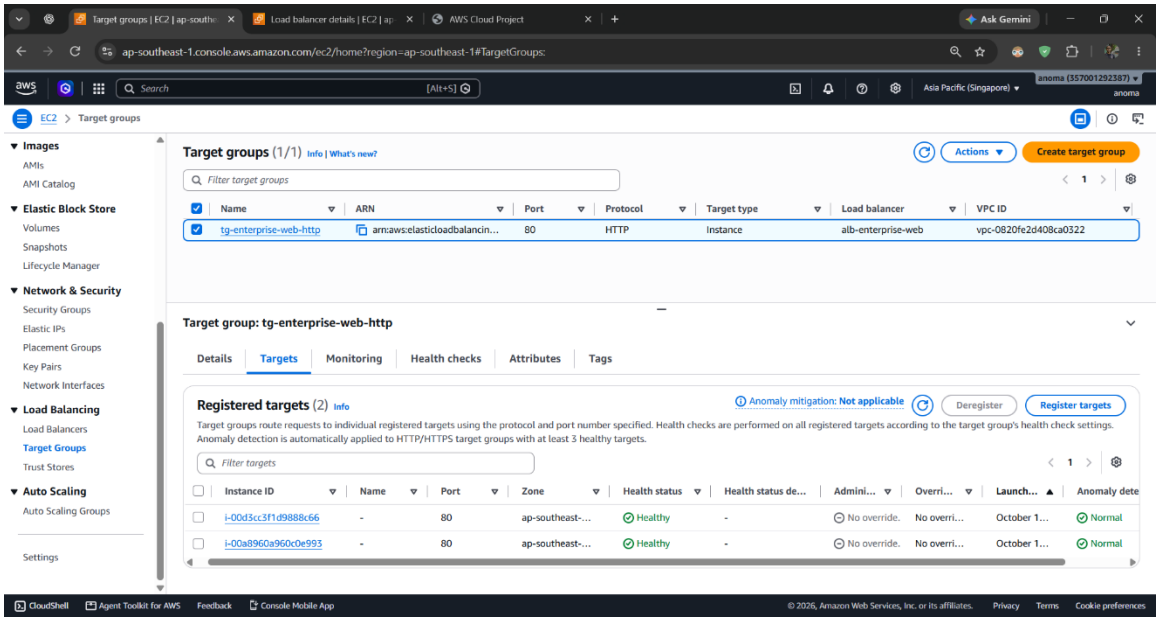


9.18 Deploy Auto Scaling Group

Created asg-enterprise-web from lt-enterprise-web, associated it with both private subnets and the target group, set desired/min/max capacity to 2 and enabled Elastic Load Balancing health checks (180-second grace period).

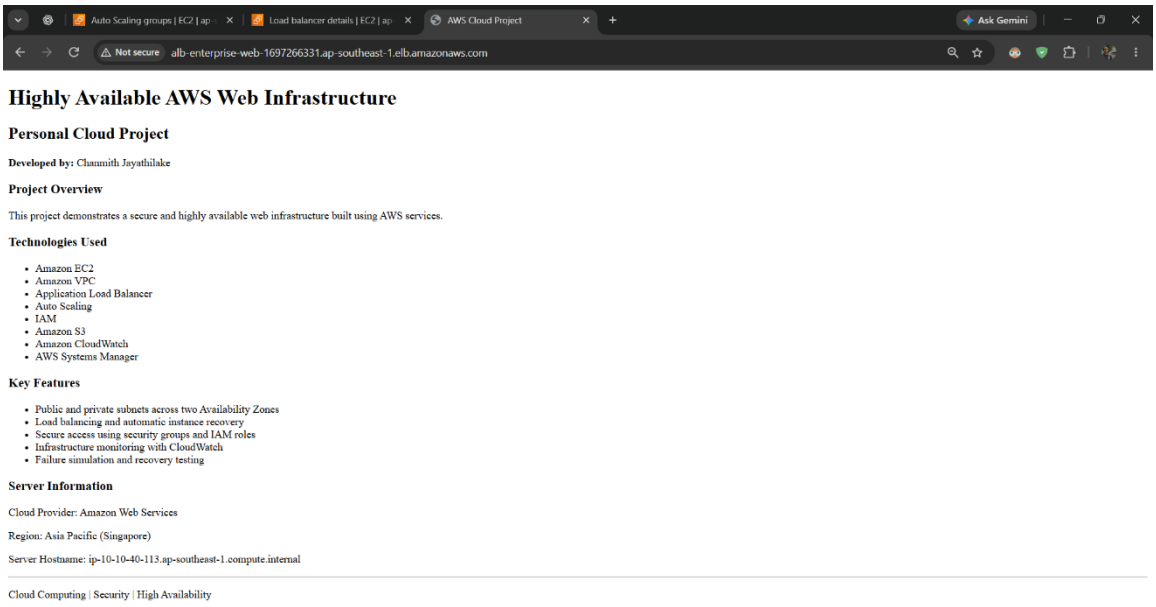


9.19 Two Healthy Targets (EC2s)

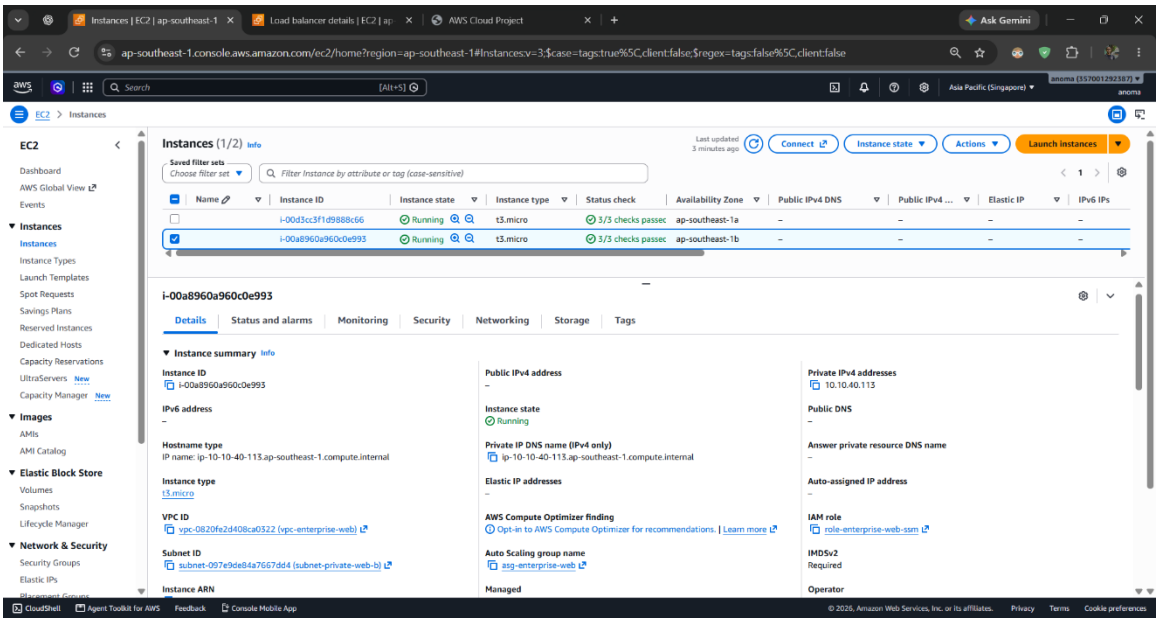


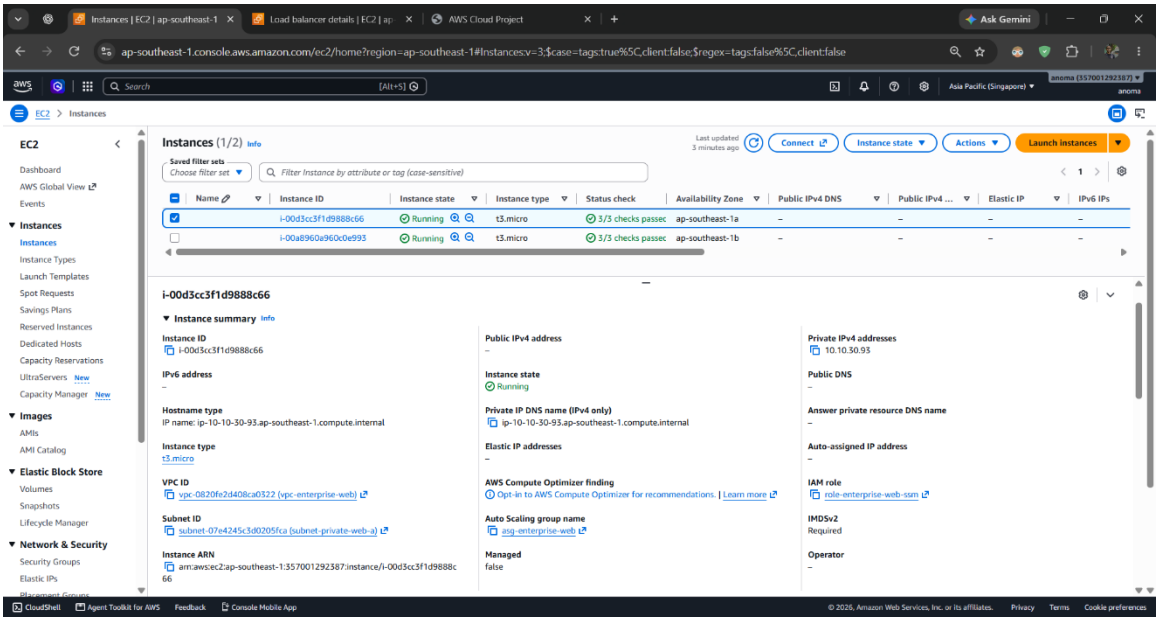
9.20 Validate Web Access and Instance Isolation

URL: http://alb-enterprise-web-1697266331.ap-southeast-1.elb.amazonaws.com/



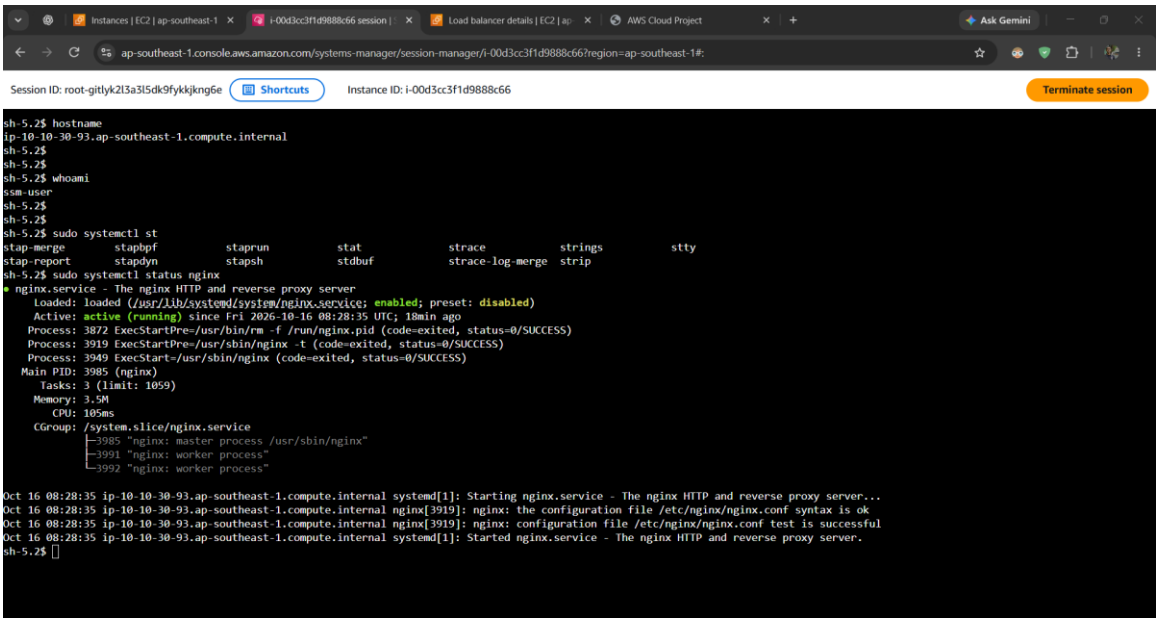
9.21 Verifying EC2 Instances have No Public IP Addresses

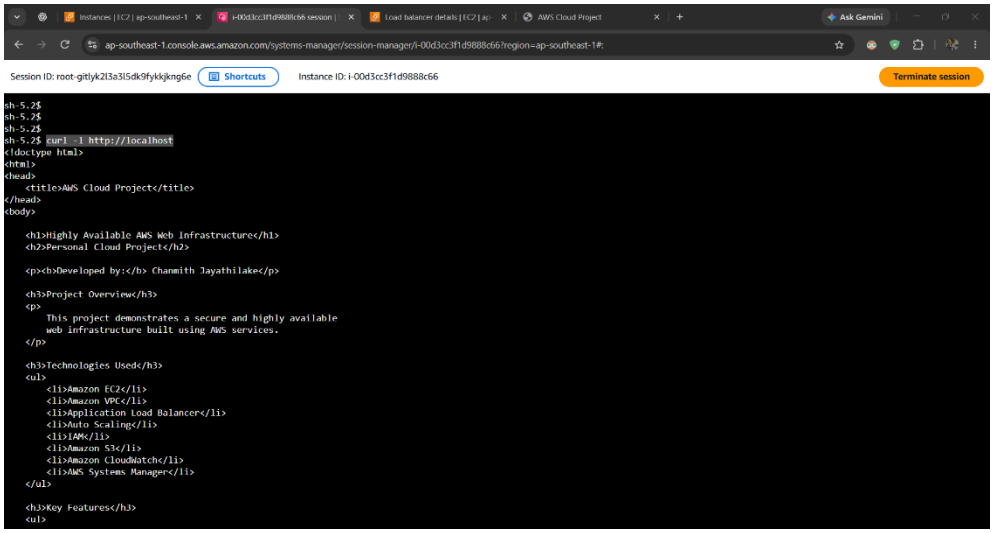




9.22 Verify AWS Systems Manager access

Used Session Manager to connect to a private managed instance without inbound SSH. Examined host name, Nginx service status and localhost HTTP response. (Both EC2 Instances)



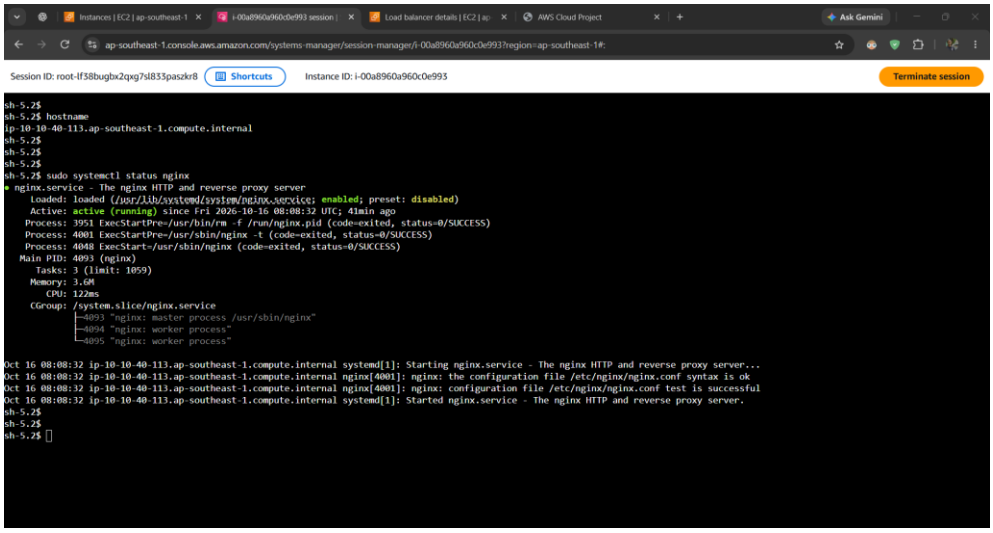

Session ID: root-gitlyk2l3a3l5dk9fykkjkng6e
Shortcuts
Instance ID: i-00d3cc3f1d9888c66
Terminate session
sh-5.2$
sh-5.2$
sh-5.2$
sh-5.2$ curl -l http://localhost
<!doctype html>
<html>
<head>
<title>AWS Cloud Project</title>
</head>
<body>
<h1>Highly Available AWS Web Infrastructure</h1>
<h2>Personal Cloud Project</h2>
Developed by: Chanmith Jayathilake
<h3>Project Overview</h3>

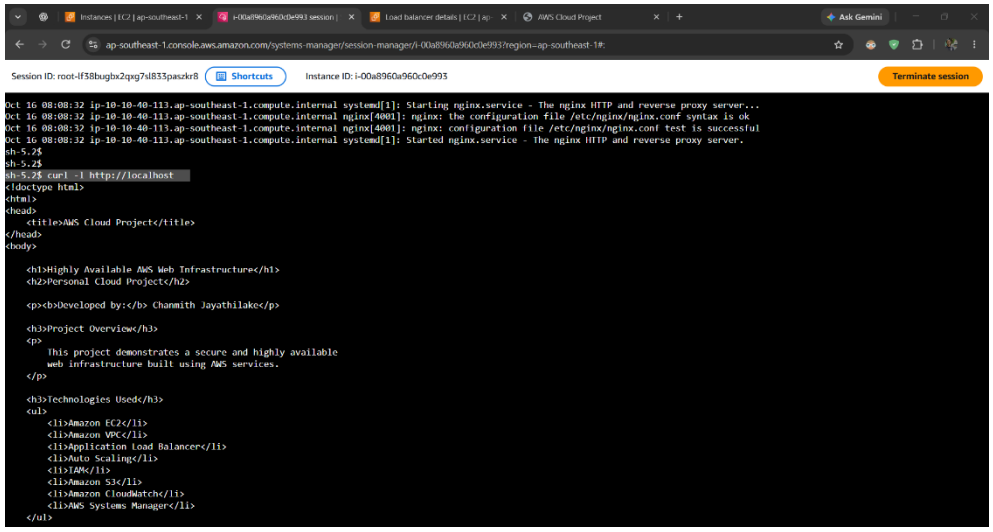
This project demonstrates a secure and highly available
web infrastructure built using AWS services.

<h3>Technologies Used</h3>

Amazon EC2
Amazon VPC
Application Load Balancer
Auto Scaling
IAM
Amazon S3
Amazon CloudWatch
AWS Systems Manager

<h3>Key Features</h3>


Session ID: root-lf38bugbx2qxg7sl833paszkr8
Shortcuts
Instance ID: i-00a8960a960c0e993
Terminate session
sh-5.2$
sh-5.2$ hostname
ip-10-10-40-113.ap-southeast-1.compute.internal
sh-5.2$
sh-5.2$
sh-5.2$
sh-5.2$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
Active: active (running) since Fri 2026-10-16 08:08:32 UTC; 41min ago
Process: 3951 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
Process: 4001 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
Process: 4048 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
Main PID: 4093 (nginx)
Tasks: 3 (limit: 1059)
Memory: 3.6M
CPU: 122ms
CGroup: /system.slice/nginx.service
├─4093 "nginx: master process /usr/sbin/nginx"
├─4094 "nginx: worker process"
└─4095 "nginx: worker process"
Oct 16 08:08:32 ip-10-10-40-113.ap-southeast-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Oct 16 08:08:32 ip-10-10-40-113.ap-southeast-1.compute.internal nginx[4001]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Oct 16 08:08:32 ip-10-10-40-113.ap-southeast-1.compute.internal nginx[4001]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Oct 16 08:08:32 ip-10-10-40-113.ap-southeast-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
sh-5.2$
sh-5.2$
sh-5.2$


Session ID: root-lf38bugbx2qxg7sl833paszkr8
Shortcuts
Instance ID: i-00a8960a960c0e993
Terminate session
Oct 16 08:08:32 ip-10-10-40-113.ap-southeast-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Oct 16 08:08:32 ip-10-10-40-113.ap-southeast-1.compute.internal nginx[4001]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Oct 16 08:08:32 ip-10-10-40-113.ap-southeast-1.compute.internal nginx[4001]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Oct 16 08:08:32 ip-10-10-40-113.ap-southeast-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
sh-5.2$
sh-5.2$
sh-5.2$ curl -l http://localhost
<!doctype html>
<html>
<head>
<title>AWS Cloud Project</title>
</head>
<body>
<h1>Highly Available AWS Web Infrastructure</h1>
<h2>Personal Cloud Project</h2>
Developed by: Chanmith Jayathilake
<h3>Project Overview</h3>

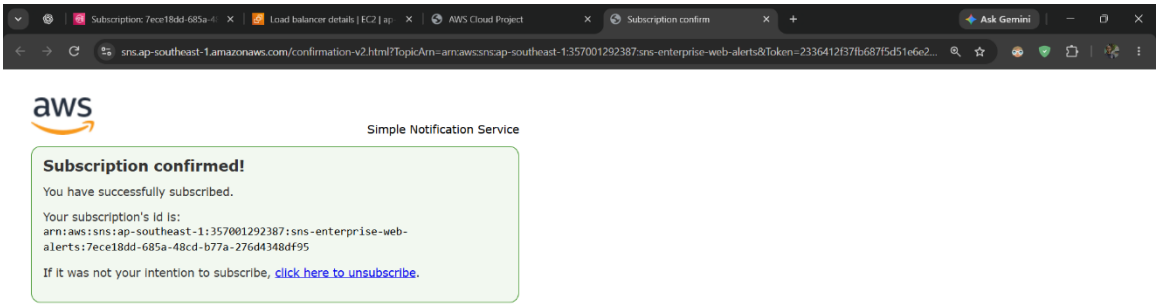
This project demonstrates a secure and highly available
web infrastructure built using AWS services.

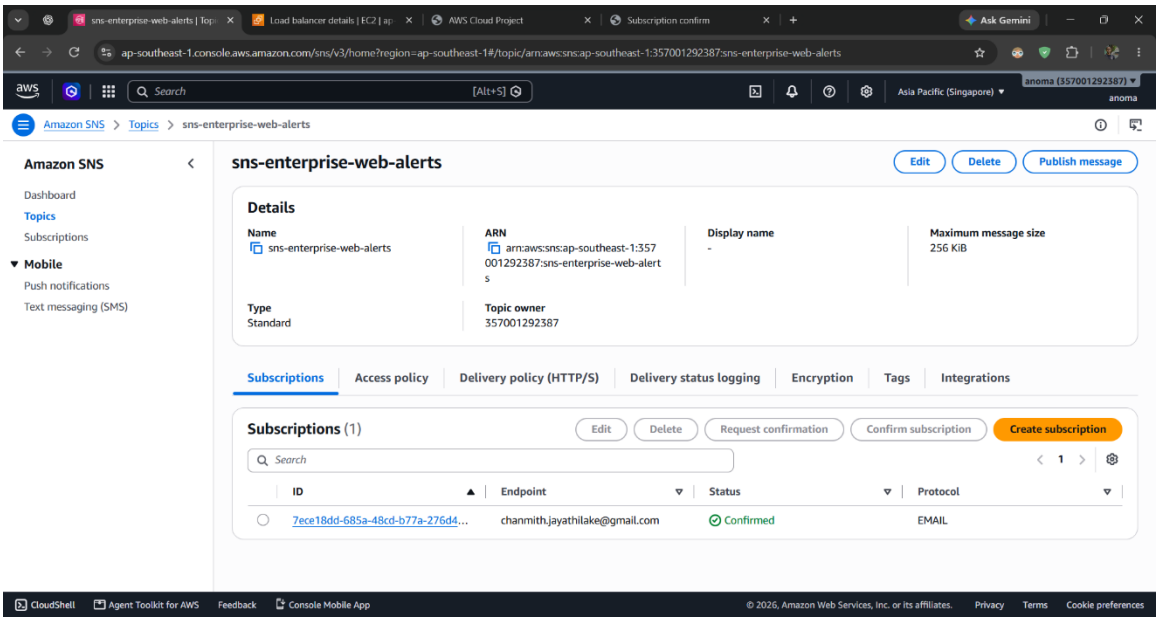
<h3>Technologies Used</h3>

Amazon EC2
Amazon VPC
Application Load Balancer
Auto Scaling
IAM
Amazon S3
Amazon CloudWatch
AWS Systems Manager

9.23 Configure SNS email alerting

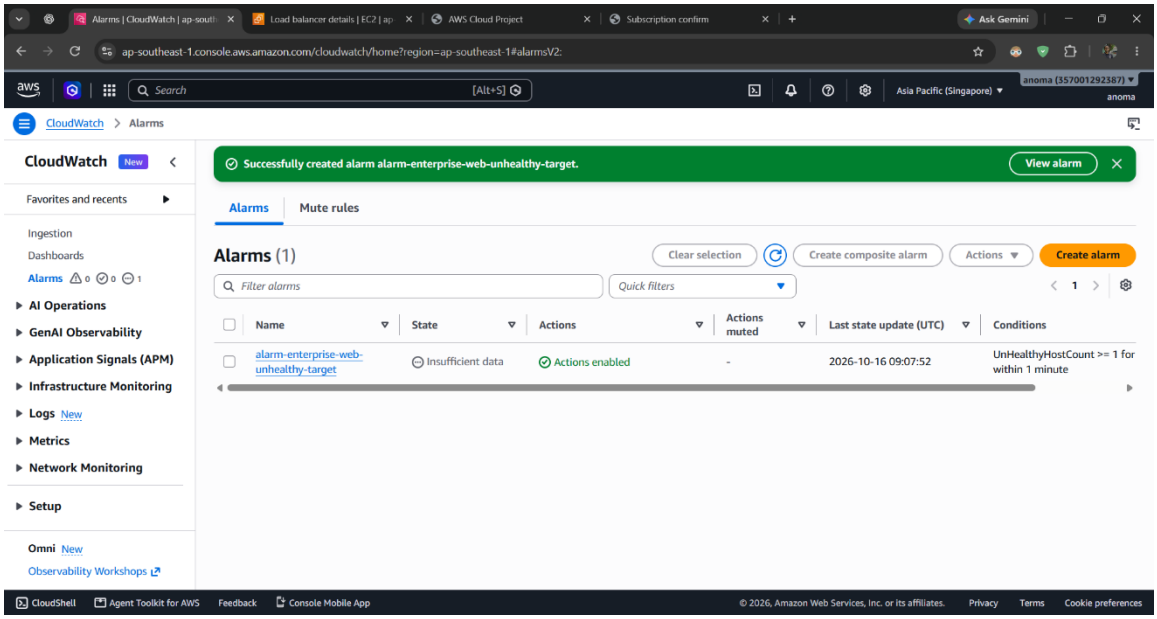
Created SNS topic sns-enterprise-web-alerts and a confirmed email subscription.

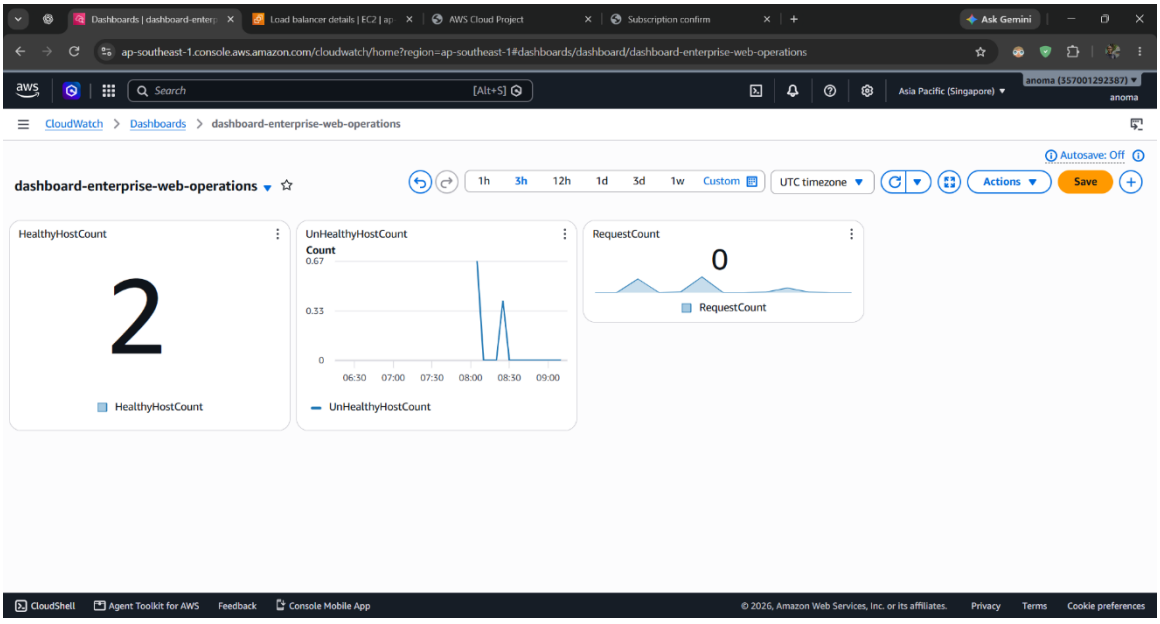




9.24 Configure CloudWatch Monitoring and Dashboard

Configured CloudWatch alarm alarm-enterprise-web-unhealthy-target for ALB UnHealthyHostCount >= 1; created dashboard-enterprise-web-operations.

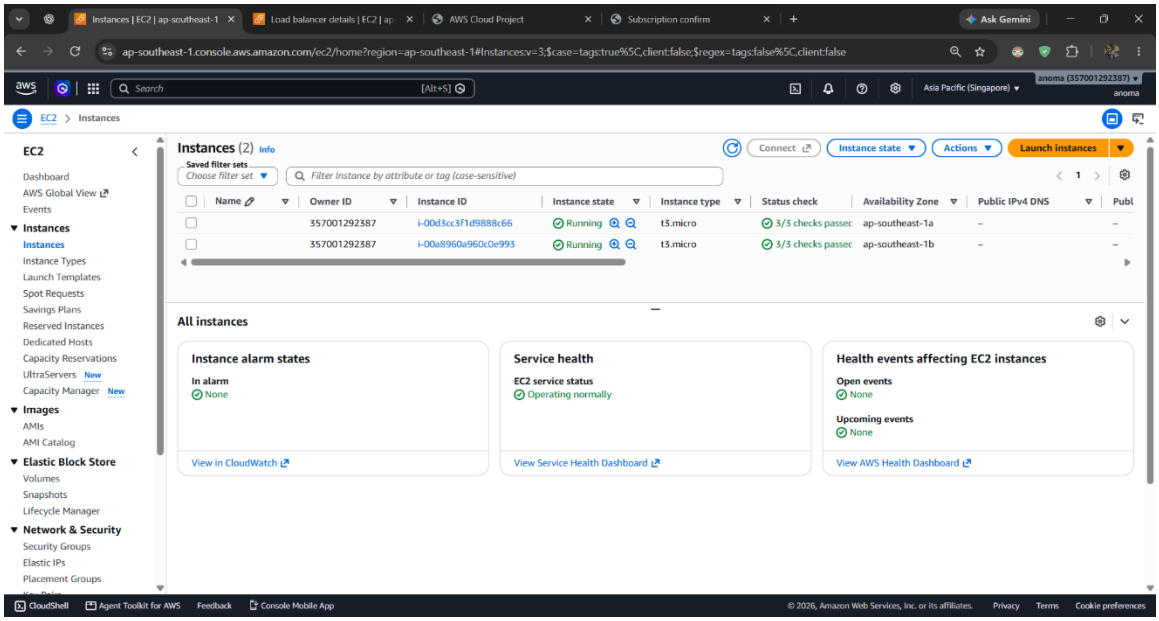




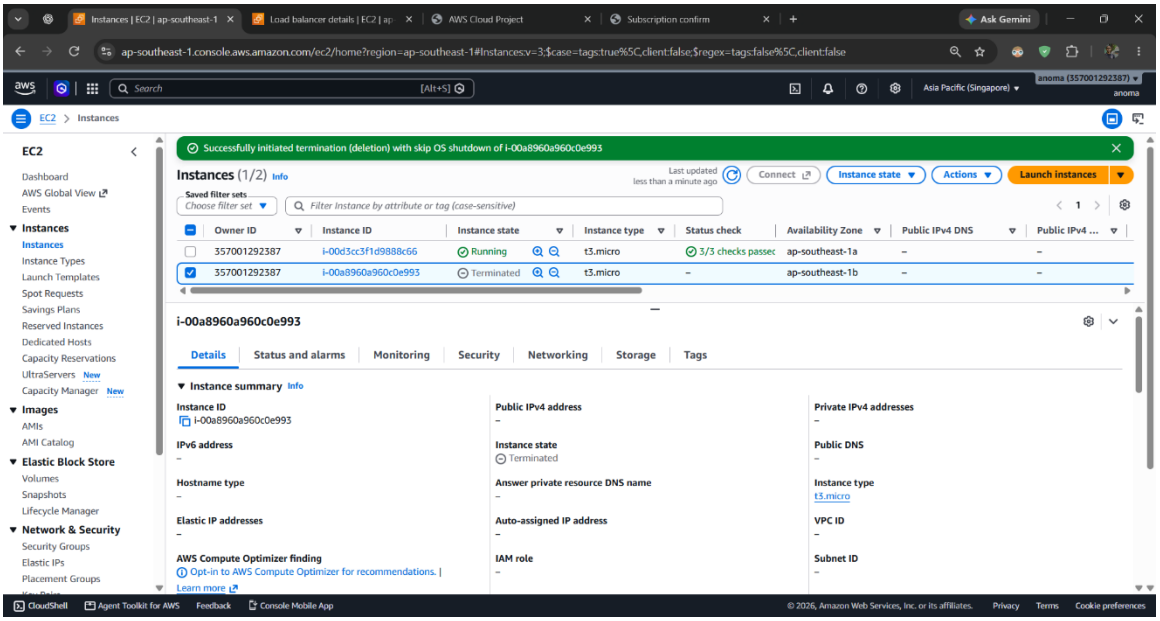
9.25 Auto Scaling and Automated Instance Recovery Testing

Identified one ASG-managed instance and terminated it as a test. Reviewed Auto Scaling Activity for replacement and checked whether service targets returned to the intended healthy count. Insert actual timestamps, instance IDs and outcome below.

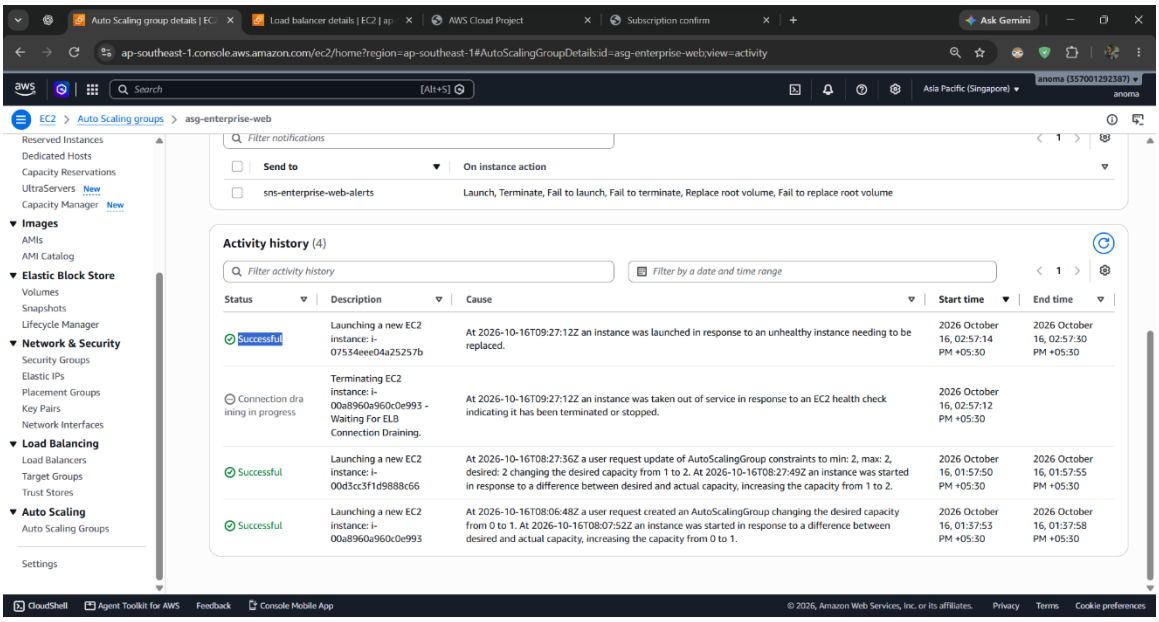
Capturing the target group view showing 2 Healthy, and both EC2 instance IDs.



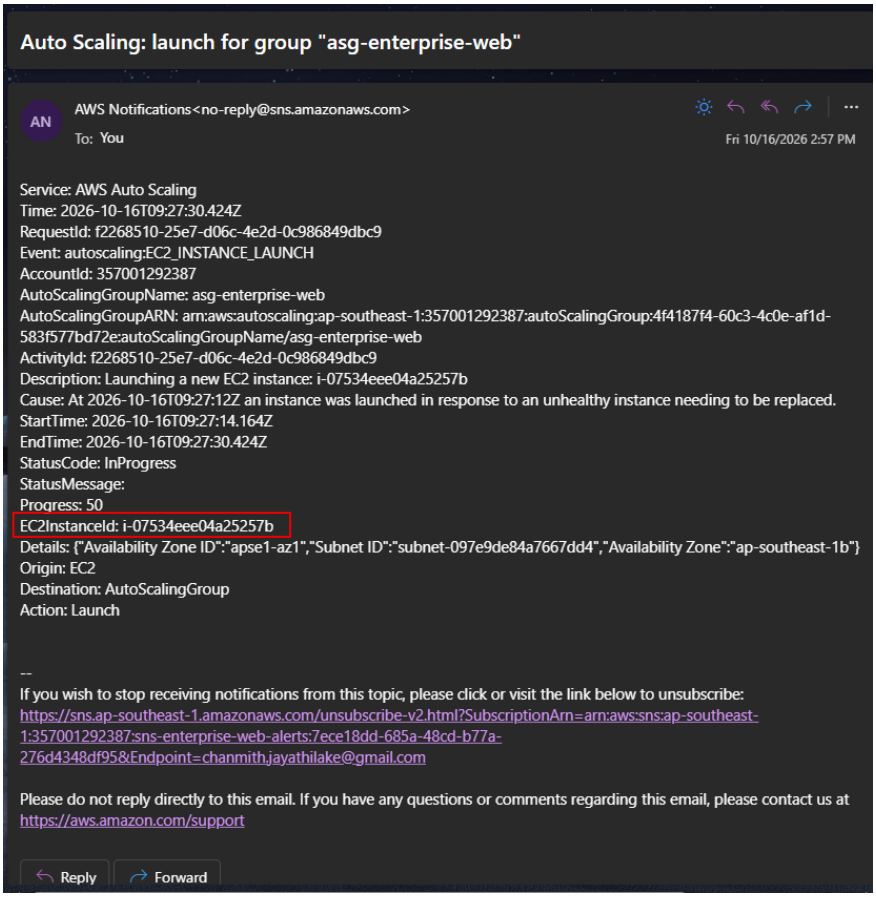
Forcefully terminate the EC2 Instance (ID: i-00a8960a960c0e993)



Launching the New EC2 Instance instead of Corrupted Instance (ID: i-07534eee04a25257b)



Gettings Email Alerts (ID: i-07534eee04a25257b)



10. REFERENCES

[1] Amazon VPC User Guide - Create a VPC.
https://docs.aws.amazon.com/vpc/latest/userguide/create-vpc.html

[2] Amazon EC2 Auto Scaling - Configure load balancing.
https://docs.aws.amazon.com/autoscaling/ec2/userguide/autoscaling-load-balancer.html

[3] Amazon EC2 Auto Scaling - Health checks.
https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto scaling-health-checks.html

[4] Elastic Load Balancing - Application Load Balancers.
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

[5] Systems Manager - Session Manager. https://docs.aws.amazon.com/systems-manager/latest/userguide/session manager.html

[6] Amazon CloudWatch - CloudWatch alarms.
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html

[7] Amazon SNS - Email notifications. https://docs.aws.amazon.com/sns/latest/dg/sns-email-notifications.html

[8] AWS Pricing Calculator. https://calculator.aws/